Real-Time Indian Sign Language Translation

A PROJECT REPORT

Submitted by

JAGADISH V [RA2212704010025]

KISHOR S [RA2212704010024]

PRANAV PRAMOD [RA2212704010010]

Under the Guidance of

Dr. Sinthuja

Assistant Professor, Department of Data Science and Business Systems

in partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE ENGINEERING

with specialization in

DATA SCIENCE AND BUSINESS SYSTEMS



DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR- 603 203

MAY 2026

Department of Data Science and Business Systems

SRM Institute of Science & Technology

Own Work Declaration Form*

To be completed by the student for all assessments

Degree/ Course : **M.Tech (Int) / 21CSP401L – Major Project**

Evaluation Student Name : **JAGADISH ; KISHOR S ; PRANAV PRAMOD**

Registration Number : **RA2212704010025, RA2212704010024,RA2212704010010**

Title of Work : **Real-Time Indian Sign Language Translation**

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism**, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is my / our own except where indicated, and that We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

I understand that any false claim for this work will be penalized in

DECLARATION:		
We are aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above.		
JAGADISH V	KISHOR S	PRANAV PRAMOD
RA2212704010025	RA2212704010024	RA2212704010010

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 18CSP462L – Project Evaluation - II report titled **"Urban Green Space Optimizer"** is the bonafide work of **"MUGASH PRIYAN U [RA2212704010028], ANANTHANARAYANA M [RA2212704010016]"** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. C. TAMIZHSELVAN
Supervisor,
Assistant Professor,
Department of DSBS,
SRMIST,KTR.

SIGNATURE

DR. V. KAVITHA
Professor & Head ,
Department of DSBS,
SRMIST,KTR.

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson, School of Computing and **Dr. C. Lakshmi,** Professor and Associate Chairperson, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. V. Kavitha**, Professor, Department of Data Science and Business Systems, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr. Prabhu Kavin**, Department of Data Science and Business Systems, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. C.Tamizhselvan** , Department of Data Science and Business Systems, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under his / her mentorship. He / She provided us with the freedom and support to explore the research topics of our interest. His / Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff and students of Data Science and Business Systems, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement.

JAGADISH V [RA2212704010025]

KISHOR S [RA2212704010024]

PRANAV PRAMOD [RA2212704010010]

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

Kattankulathur, Chennai — 603203

Department of Data Science and Business Systems

PROJECT REPORT

ISL-RT-AI1

Real-Time Indian Sign Language Translation

Using Graph Neural Networks, Dilated Temporal Convolutions, and CTC Decoding

Jagadish V	Kishor	Pranav Pramod
RA2212704010025	RA2212704010024	RA2212704010010

Under the Guidance of

Dr. K. Shantha Kumari

Associate Professor, Department of Data Science and Business Systems

Academic Year 2024 – 2025

1. Abstract

ISL-RT-AI1 is a complete, end-to-end real-time Indian Sign Language translation system built and deployed during the academic year 2024–25 by a three-member undergraduate team at SRMIST Kattankulathur. The system accepts live webcam video or uploaded sign language clips and produces English text output in real time, running at 25–30 frames per second on a consumer laptop GPU without any specialist hardware.

The central technical contribution is the representation of the 553 human body landmarks detected per frame by MediaPipe Holistic as a spatial graph, processed by a Graph Neural Network (GNN). This graph encoding, combined with dilated temporal convolutions, a Transformer sequence encoder, and Connectionist Temporal Classification (CTC) decoding, achieves a Word Error Rate of 0.0458 and a Word Accuracy of 95.4% on a 241-label ISL vocabulary — a 68% WER reduction compared to an identical system without graph encoding.

The project also delivers a full-stack web application: a FastAPI backend serving model inference via REST and WebSocket endpoints, and a React frontend with three functional tabs — Live Camera translation, Video Upload translation, and Text-to-Sign display.

2. Project Overview

2.1 Problem Statement

India has over 63 million people with significant hearing impairment. The majority rely on Indian Sign Language as their primary communication medium. Despite this large population, no real-time, hardware-free ISL translation tool is available for everyday deployment at hospitals, schools, or public offices. Existing academic systems either require specialist depth cameras or data gloves, or operate only on tiny isolated-word vocabularies with no sentence-level recognition.

2.2 Objectives

- Build a real-time ISL recognition system that works with a standard webcam only
- Achieve Word Accuracy above 90% on a vocabulary of at least 200 ISL labels
- Demonstrate that graph-structured body encoding outperforms flat-vector baselines
- Deploy the system as a full-stack web application accessible through a browser
- Document all engineering findings for reproducibility and future extension

2.3 Scope

Aspect	In Scope	Out of Scope
Language	Indian Sign Language (ISL)	ASL, BSL, other sign languages
Input	Webcam video / uploaded MP4	Sensor gloves, depth cameras
Output	English text (word / phrase)	Spoken audio synthesis
Hardware	Consumer laptop + GPU	Server GPU, cloud deployment
Vocabulary	241 labels, sentence-level	Unlimited open vocabulary
Mode	Real-time + batch upload	Mobile app (future work)

2.4 Team and Roles

Member	Roll Number	Primary Role
Jagadish V	RA2212704010025	ML Architecture, GNN Training, Backend API
Kishor	RA2212704010024	Data Collection, Preprocessing Pipeline, Evaluation
Pranav Pramod	RA2212704010010	Frontend Development, System Integration, Testing

3. System Architecture

3.1 High-Level Pipeline

The complete system processes video in six sequential stages:

01	**Landmark Extraction**	MediaPipe Holistic extracts 553 body landmark (x,y,z) coordinates per frame → 1,659-dimensional feature vector
02	**Normalisation**	Per-sequence z-score normalisation applied to non-padding frames only, clamped std > 1e-6
03	**Graph Neural Network**	2-layer GCN processes 553-node anatomical body graph per frame → 8,848-dimensional spatial encoding
04	**Temporal CNN**	3 dilated causal convolution blocks (dilation 1, 2, 4) capture motion at local, medium and extended scales
05	**Transformer Encoder**	2-layer multi-head self-attention captures long-range dependencies across the full 120-frame window
06	**CTC Decoding**	Greedy CTC decoding collapses per-frame class distribution → final word sequence

3.2 Landmark Graph Structure

Each frame is represented as a graph G = (V, E) where V is the set of 553 landmark nodes and E is the set of anatomical edges:

Body Region	Nodes	Edge Type
Face Mesh (with iris)	478	Mesh triangulation edges + iris ring
Left Hand	21	5 finger kinematic chains + palm
Right Hand	21	5 finger kinematic chains + palm
Pose Skeleton	33	Upper body joint connections
Total	553	1,659 features (553 × 3 coordinates)

NOTE When refine_face_landmarks=True is set in MediaPipe Holistic, the face landmark count increases from 468 to 478 (iris tracking enabled), raising the total feature dimension from 1,629 to 1,659. Systems that hard-code 1,629 silently discard all face data. ISL-RT-AI1 explicitly validates and uses 1,659 dimensions.

3.3 Model Architecture Detail

Component	Layer / Parameter	Value / Description
Input	Feature dimension	1,659 per frame (553 nodes × 3)
Input	Max sequence length	120 frames (zero-padded)
GNN Layer 1	Graph convolution	Lin(1659→16), degree-norm adjacency
GNN Layer 1	Activation	ReLU + Residual connection
GNN Layer 2	Graph convolution	Lin(16→16), same adjacency
GNN Output	Flattened per frame	553 × 16 = 8,848 dimensions
TCN Block 1	Dilated causal conv	Dilation = 1, kernel = 3
TCN Block 2	Dilated causal conv	Dilation = 2, kernel = 3
TCN Block 3	Dilated causal conv	Dilation = 4, kernel = 3
Transformer	Encoder layers	2 layers, multi-head self-attention
CTC Head	Output classes	241 vocabulary + 1 blank token = 242
Optimiser	AdamW	lr=1e-4, weight decay=1e-2
Gradient clip	Max norm	1.0
Mixed precision	AMP	torch.cuda.amp, auto-scaling

4. Dataset and Preprocessing

4.1 Dataset Statistics

4,317	3,453	864	241
Total Videos across 241 labels	**Training Split** 80%, seed=42	**Validation Split** 20%, stratified	**Vocabulary** words + phrases

4.2 Vocabulary Categories

- Single-concept signs: bear, volcano, uncle, wife, mother, father, teacher, doctor, ...
- Action signs: eat, drink, sleep, walk, run, write, study, ...
- Sentence-level phrases: why are you crying, where are you from, what is your name, ...
- Category sizes range from 1 sample (deficient) to 20+ samples (well-covered)

4.3 Preprocessing Pipeline

Each video is processed as follows:

1. Load video using OpenCV, read frame by frame
2. Pass each frame to MediaPipe Holistic with refine_face_landmarks=True
3. Extract 553 landmarks per frame → 1,659-dimensional vector
4. Validate: discard frame if std < 0.01 (extraction failure)
5. If sequence length > 120: uniformly subsample to 120 frames
6. If sequence length < 120: zero-pad to 120 frames, record actual length
7. Apply per-sequence z-score normalisation over actual (non-padding) frames only
8. Save as .npy file in data/landmarks/ for fast training loading

4.4 Critical Bug: Multiprocessing Corruption

BUG FOUND

When MediaPipe Holistic is called from Python's multiprocessing.Pool child processes on Windows, it returns all-zero arrays silently. This corrupted 94.3% of initially extracted landmarks. Detection method: check if std across all frames < 0.01. Fix: migrate to single-process extraction. All data was re-extracted after this fix.

```
# Corruption detection
def is_corrupt(landmarks: np.ndarray) -> bool:
    return landmarks.std() < 0.01

# Per-sequence normalisation (non-padding frames only)
def normalize(seq: np.ndarray) -> np.ndarray:
    mask = np.any(seq != 0, axis=1)      # True for actual frames
```

```
    if mask.sum() == 0: return seq
    valid = seq[mask]
    mean  = valid.mean(axis=0)
    std   = valid.std(axis=0)
    std[std < 1e-6] = 1.0               # prevent division by zero
    seq[mask] = (seq[mask] - mean) / std
    return seq
```

5. Training Strategy

5.1 Two-Phase GNN Training

A key engineering challenge was adding the GNN module to an already-pretrained temporal backbone without destabilising the existing weights. Naive joint training from epoch 1 caused catastrophic gradient interference. The solution was a two-phase strategy:

Phase	Epochs	Learning Rate	Layers Updated	Purpose
Baseline (no GNN)	50	3e-4	TCN + Transformer + CTC	Pretrain backbone
Phase 1 (GNN warm-up)	10	1e-3	GNN layers only (frozen backbone)	Initialise GNN weights
Phase 2 (full fine-tune)	40	1e-4	All layers unfrozen	Joint optimisation

5.2 Training Configuration

Parameter	Value
Hardware	ASUS TUF A15, NVIDIA RTX 3050 4GB, AMD Ryzen, 16GB RAM
Batch size	16
DataLoader workers	6
Warmup schedule	5-epoch linear warmup, then linear decay
Gradient clipping	max_norm = 1.0
Mixed precision	torch.cuda.amp with GradScaler
Baseline duration	~15 minutes (50 epochs, ~17s/epoch)
GNN fine-tune	~7 minutes (50 more epochs)
Best model saved	best_cslt_gnn.pth (lowest validation WER)

5.3 GNN Stability Fixes

Several numerical issues were encountered during GNN training and resolved:

- Exploding gradients: hidden_dim=64 caused loss=500. Fix: reduce hidden_dim to 16
- Xavier initialisation: gain=0.1 (not default 1.0) + divide by sqrt(553)
- Wrong node count: initial code used num_nodes=543 (without iris). Fix: 553
- ReLU placement: ReLU after LayerNorm killed gradients. Fix: ReLU before residual add
- CTC input lengths: hardcoded to 120 caused CTC loss=779. Fix: pass actual_len

```
class GNNEncoder(nn.Module):
    def __init__(self, in_dim=1659, hidden=16, num_nodes=553):
        super().__init__()
```

```
        self.gc1  = GraphConv(in_dim,  hidden)
        self.gc2  = GraphConv(hidden,  hidden)
        self.proj = nn.Linear(in_dim,  hidden)    # residual projection
        # Xavier init with gain=0.1
        for p in self.parameters():
            if p.dim() > 1:
                nn.init.xavier_uniform_(p, gain=0.1)

    def forward(self, x, adj):
        # x: (B, T, N, F)
        h = F.relu(self.gc1(x, adj))              # ReLU BEFORE residual
        h = h + self.proj(x)                      # residual connection
        h = self.gc2(h, adj) / (553 ** 0.5)       # scale by 1/sqrt(N)
        return h.flatten(-2)                      # (B, T, N*hidden)
```

6. Experimental Results

6.1 Overall Performance

0.0458	95.4%	88.9%	25–30
Word Error Rate	**Word Accuracy**	**Exact Match**	**FPS**
↓ 68% vs baseline	↑ from 85.7%	↑ from ~75%	RTX 3050 4GB

6.2 Baseline vs ISL-RT-AI1 Comparison

Metric	Baseline (No GNN)	ISL-RT-AI1 (GNN)	Improvement
Word Error Rate	0.1432	**0.0458**	−68.0%
Word Accuracy	85.7%	**95.4%**	+9.7 pp
Exact Match Accuracy	~75%	**88.9%**	+13.9 pp
Best Val WER (training)	0.1432	0.0633	−55.8%
Training Time	~15 min	~22 min	+47% (acceptable)
Inference Speed	25 FPS	25–30 FPS	≈ same

6.3 Per-Label Analysis

Labels achieving 100% Exact Match Accuracy on the held-out validation set:

✓ umbrella	✓ uncle	✓ vegetables
✓ volcano	✓ wife	✓ writer
✓ wrong	✓ why are you angry	✓ why are you crying

FINDING All zero-accuracy labels have 1–3 training samples. This is a data coverage problem, not a model problem. Fix: collect minimum 15 videos per label for the 31 deficient categories. Labels with 10+ samples consistently achieve 100% accuracy.

6.4 Hypothesis Testing

- Paired t-test across 20 inference runs per label, $\alpha = 0.05$
- Null hypothesis rejected for 9 of 10 highest-traffic categories ($p < 0.05$, $t > 2.1$)
- Cronbach Alpha across 6 architecture components: $\alpha = 0.87$ (high internal consistency)
- Conclusion: GNN improvement is statistically significant, not random variation

7. Implementation Details

7.1 Project Structure

```
ISL-RT-AI1/
├── models/
│   ├── cslt_model.py        # Main model: GNN + TCN + Transformer + CTC
│   └── gnn.py               # GNN encoder (GraphConv layers)
├── training/
│   ├── train.py             # Two-phase training script
│   ├── dataset.py           # ISLDataset, collate_fn, actual_len tracking
│   ├── tokenizer.py         # build_vocab(), word_to_idx, idx_to_word
│   └── evaluate.py          # WER, exact match, per-label accuracy, charts
├── preprocessing/
│   └── save_landmarks.py    # Single-process MediaPipe extraction
├── inference/
│   └── realtime_translation.py  # Webcam + motion gate + TTS
├── tts/
│   └── speak.py             # Queue-based subprocess TTS (pyttsx3 fix)
├── backend/
│   └── main.py              # FastAPI: /health, /api/vocab, /api/translate,
/ws/stream
├── frontend/
│   └── src/App.jsx          # React: Live Camera, Upload Video, Text→Sign tabs
├── data/
│   ├── landmarks/           # .npy files (extracted landmarks)
│   ├── videos/              # Original .mp4 recordings
│   └── labels.csv           # label → video mapping
├── best_cslt_gnn.pth        # Best GNN model weights
└── best_cslt_model.pth      # Best baseline model weights
```

7.2 Backend API — FastAPI

Endpoint	Method	Description
GET /health	HTTP GET	Returns status, device (cuda/cpu), vocab_size
GET /api/vocab	HTTP GET	Returns full 241-item vocabulary list
POST /api/translate	HTTP POST	Upload video file → MediaPipe → inference → prediction JSON
WS /ws/stream	WebSocket	Send base64 JPEG frames → receive prediction + signing status

7.3 Frontend — React Application

Tab	Description	Key Feature
📷 Live Camera	Webcam → WebSocket → real-time prediction	FPS counter, signing/idle status dot, prediction history
📁 Upload Video	Drag-drop video → POST → text result	Video preview, frame count, confidence display
✍ Text → Sign	Type words → sign image cards displayed	Auto-play mode, vocab check, placeholder for unknown words

7.4 TTS — Text to Speech (Subprocess Fix)

BUG FIXED pyttsx3's runAndWait() fails silently on the 2nd call in background threads on Windows. Fix: spawn a fresh Python subprocess for each speech call instead of reusing the engine.

```
# speak.py — subprocess-based TTS (bypasses pyttsx3 thread bug)
import subprocess, queue, threading

_q = queue.Queue(maxsize=1)

def _worker():
    while True:
        text = _q.get()
        if text is None: break
        subprocess.run([
            'python', '-c',
            f'import pyttsx3; e=pyttsx3.init(); e.say({repr(text)}); e.runAndWait()'
        ], timeout=10)

threading.Thread(target=_worker, daemon=True).start()

def speak(text: str):
    try: _q.put_nowait(text)    # skip if busy
    except queue.Full: pass
```

8. All Bugs Discovered and Resolved

#	Bug	Symptom	Fix
1	MediaPipe multiprocessing	All-zero landmarks, 94.3% files corrupt	Single-process extractor
2	Wrong feature dimension	Tensor shape mismatch (1629 vs 1659)	Set EXPECTED_FEATURES=1659
3	CTC input_lengths hardcoded	CTC loss = 779, training diverged	Pass actual_len per sample
4	GNN exploding gradients	Loss = 500 on first epoch	hidden_dim=16, Xavier gain=0.1
5	Wrong num_nodes (543 vs 553)	GNN adjacency matrix mismatch	Fix to 553 everywhere
6	ReLU after LayerNorm	Killed negative activations, slow learning	ReLU on conv before residual
7	pyttsx3 thread failure	TTS silent after first call	Fresh subprocess per call
8	Normalisation on padding	Distorted statistics, low accuracy	Normalise actual frames only

9. Conclusions and Future Work

9.1 Conclusions

ISL-RT-AI1 demonstrates that a GNN-based body graph encoder, when combined with dilated temporal convolutions and Transformer attention, achieves substantially better sign language recognition than flat-vector approaches. The 68% WER reduction over the baseline confirms the value of encoding anatomical spatial structure explicitly. The system operates in real time on consumer hardware and is fully accessible through a browser-based interface.

Four broader lessons from the project: (1) the spatial encoding of landmarks is more important than the temporal model complexity; (2) preprocessing quality control — detecting and discarding corrupt extractions — had a larger impact on accuracy than any architectural change; (3) a two-phase training strategy is necessary when adding an uninitialised module to a pretrained backbone; and (4) the accuracy ceiling for any label is determined by the number of training samples, not the model.

9.2 Future Work

Direction	Description	Expected Impact
Dataset expansion	Collect 15+ videos for all 31 deficient labels	Zero-accuracy labels → ~90%
Beam search CTC	Replace greedy with beam search + n-gram LM	Further 10–15% WER reduction
Sign boundary detection	Replace fixed 120-frame window with dynamic segmentation	Better long-sign handling
ONNX export	Export model to ONNX for Android/iOS deployment	Mobile accessibility
Temporal augmentation	Random speed perturbation during training	Better signer generalisation
Multi-signer dataset	Record all labels from 3+ different signers	Improved robustness

10. References

[1] World Health Organization, "Deafness and hearing loss," WHO Fact Sheet, Geneva, 2023.

[2] S. S. Rautaray and A. Agrawal, "Vision based hand gesture recognition for human computer interaction: a survey," Artif. Intell. Rev., vol. 43, pp. 1–54, 2015.

[3] N. C. Camgoz, O. Koller, S. Hadfield, R. Bowden, "Sign language transformers," IEEE/CVF CVPR, 2020.

[4] S. Yan, Y. Xiong, D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," AAAI, 2018.

[5] A. Graves, S. Fernandez, F. Gomez, J. Schmidhuber, "Connectionist temporal classification," ICML, 2006.

[6] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

[7] H. Zhou et al., "Spatial-temporal multi-cue network for continuous sign language recognition," IEEE Trans. Multimed., vol. 23, 2021.

[8] A. Vaswani et al., "Attention is all you need," NeurIPS, 2017.

[9] O. Koller, J. Forster, H. Ney, "Continuous sign language recognition," Comput. Vision Image Understand., 2015.

[10] D. A. Kumar et al., "Indian sign language recognition using wearable sensors," Comput. Electr. Eng., vol. 90, 2021.